

AI STUDY BUDDY API

Requirement and Analysis Phases

Team leader name : Nithya Sree S
Team member 1 : Naseeha Begum R
Team member 2 : Niranjana S
Team member 3 : Noori Nisha H N

1. Introduction

The Requirement Phase is the first major stage in developing the **AI StudyBuddy API**. The purpose of this phase is to identify what the system should do, who will use it, what resources are required, and what limitations need to be considered.

AI StudyBuddy is designed as an intelligent educational system that assists students with learning activities such as asking questions, generating notes, creating quizzes, preparing flashcards, and organizing study activities.

The requirements are divided into:

- Functional requirements
 - Non-functional requirements
 - User requirements
 - Hardware requirements
 - Software requirements
 - API requirements
-

2. Objectives of the System

The main objectives of AI StudyBuddy are:

1. To provide students with an AI-based learning assistant.
 2. To answer educational questions.
 3. To generate study notes automatically.
 4. To generate quizzes based on selected topics.
 5. To create flashcards for revision.
 6. To maintain student learning information.
 7. To provide a simple API for communication between frontend and backend.
 8. To provide secure access to student information.
-

3. Target Users

The primary users of the system are:

Students

Students use the application to:

- Ask questions.
- Read generated notes.
- Practice quizzes.
- Review flashcards.
- Track their learning activities.

Administrator

An administrator can manage:

- Users
 - Educational content
 - System settings
 - API usage
 - Application data
-

FUNCTIONAL REQUIREMENTS

1. User Registration

The system shall allow a new student to create an account.

Input

Name
Email
Password
Course
Year

API

POST /api/auth/register

Expected Output

```
{  
  "success": true,  
  "message": "Registration successful"  
}
```

2. User Login

The system shall authenticate registered users.

POST /api/auth/login

Example request:

```
{
  "email": "student@example.com",
  "password": "password123"
}
```

The API returns an authentication token after successful login.

3. AI Question Answering

Students shall be able to send questions to the AI StudyBuddy.

POST /api/chat

Example:

```
{
  "message": "Explain artificial intelligence in simple words"
}
```

Example response:

```
{
  "success": true,
  "response": "Artificial Intelligence is the ability of computers..."
}
```

4. Note Generation

The system shall generate study notes from a topic.

POST /api/notes/generate

Example:

```
{
  "topic": "Computer Networks",
  "level": "beginner"
}
```

The AI generates concise and understandable notes.

5. Quiz Generation

Students shall be able to generate questions for practice.

POST /api/quizzes

Example:

```
{
  "topic": "Python",
  "number_of_questions": 5,
  "difficulty": "easy"
}
```

The API returns questions and multiple-choice options.

6. Flashcard Generation

The system shall create flashcards from a selected topic.

POST /api/flashcards

Example:

```
{
  "topic": "Database Management",
  "number_of_cards": 10
}
```

7. Progress Tracking

The system shall record learning activities such as:

- Quiz scores
 - Completed quizzes
 - Topics studied
 - Notes created
 - Flashcards reviewed
-

NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe the quality and performance expected from the AI StudyBuddy API.

1. Performance

The API should process requests efficiently and return responses within an acceptable amount of time.

For example:

Client → API Request → AI Service → API Response → Client

The system should avoid unnecessary processing and database operations.

2. Security

Security is an important requirement because the application handles user accounts and learning information.

The system should:

- Use HTTPS.
 - Protect passwords.
 - Use authentication tokens.
 - Validate user input.
 - Prevent unauthorized access.
 - Protect API keys.
 - Apply rate limiting.
 - Avoid exposing confidential information in error messages.
-

3. Scalability

The API should be designed so that additional users and features can be supported in the future.

For example:

Current System
↓
100 Users
↓
1,000 Users
↓
10,000+ Users

A scalable architecture allows additional servers and database resources to be added when required.

4. Reliability

The system should continue operating correctly when individual requests fail.

For example, if an AI request fails, the API should return a meaningful error:

```
{  
  "success": false,  
  "message": "AI service temporarily unavailable"  
}
```

5. Usability

The API should have clear endpoint names and consistent request and response formats.

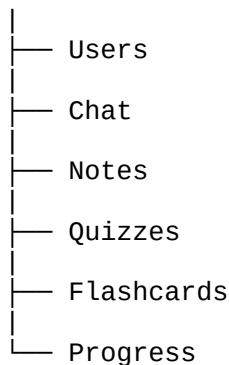
For example:

```
POST /api/chat
POST /api/notes
POST /api/quizzes
GET  /api/notes
GET  /api/profile
```

6. Maintainability

The API should be organized into separate modules.

Authentication



This makes the application easier to test, modify, and extend.

SYSTEM ANALYSIS PHASE

1. Introduction

System Analysis examines the requirements collected during the requirement phase and determines how the proposed AI StudyBuddy API should operate.

The analysis identifies:

- Inputs
 - Processing
 - Outputs
 - Users
 - Data
 - API interactions
 - System constraints
-

2. Existing System

In a traditional learning environment, students may use different applications for different activities.

For example:

Search Engine → Find Information

Notes App → Store Notes

Quiz Website → Practice Questions

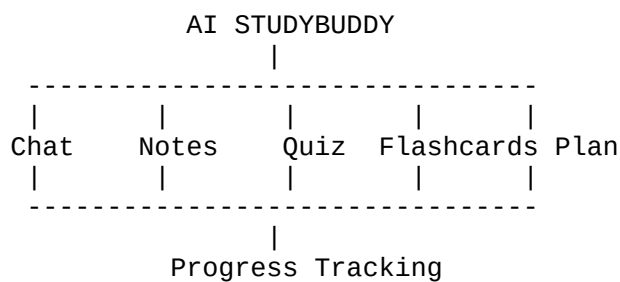
Flashcard App → Revision

Calendar → Study Planning

Using multiple applications can make learning resources difficult to organize.

3. Proposed System

The proposed AI StudyBuddy combines multiple learning functions into one system.



The backend API provides communication between the user interface, database, and AI service.

4. Input–Process–Output Analysis

Input

The system receives:

- Student questions
- Topics
- Quiz preferences
- Difficulty levels
- Number of questions
- Flashcard requirements
- User profile information

Process

The API:

1. Authenticates the user.

2. Validates the request.
3. Sends appropriate data to the AI service.
4. Processes the generated response.
5. Stores required information.
6. Sends the result to the frontend.

Output

The system produces:

- AI answers
 - Study notes
 - Quiz questions
 - Flashcards
 - Scores
 - Progress information
-

API ANALYSIS, DATA FLOW AND CONCLUSION

1. API Data Flow

The general data flow is:

```
graph TD
    Student --> Frontend_Application[Frontend Application]
    Frontend_Application --> REST_API[REST API]
    REST_API --> Authentication
    Authentication --> Business_Logic[Business Logic]
    Business_Logic --> AI_Service_Database[AI Service / Database]
    AI_Service_Database --> Response
    Response --> Frontend
    Frontend --> Student
```

2. Example API Flow

Suppose a student wants to generate a quiz.

Step 1 – Student selects topic

Topic = "Python Programming"

Step 2 – Frontend sends request

POST /api/quizzes

Step 3 – API validates request

The API checks:

- Authentication
- Topic
- Number of questions
- Difficulty

Step 4 – AI service generates questions

The backend sends the required information to the AI service.

Step 5 – API receives result

```
{
  "question": "Which keyword is used to define a function in Python?",
  "options": [
    "function",
    "define",
    "def",
    "fun"
  ]
}
```

Step 6 – Response is sent to student

The frontend displays the quiz.

3. Database Requirements

The system can maintain the following tables:

Users

Field	Description
user_id	Unique user ID
name	Student name
email	Email address
password_hash	Encrypted password
course	Course name
year	Academic year

Notes

Field	Description
note_id	Note ID
user_id	User reference
title	Note title

Field	Description
content	Note content
created_at	Creation date

Quizzes

Field	Description
quiz_id	Quiz ID
user_id	User reference
topic	Quiz topic
difficulty	Difficulty level
score	Student score
created_at	Creation date

Progress

Field	Description
progress_id	Progress ID
user_id	User reference
activity	Activity type
score	Score
completed_at	Completion date

4. System Constraints

Some possible constraints include:

- Internet connectivity may be required.
 - AI-generated responses may require external AI services.
 - API usage may be subject to rate limits.
 - AI responses may require validation before being used for educational purposes.
 - Database capacity must be considered as the number of users grows.
 - Sensitive user information must be protected.
-

5. Requirement and Analysis Conclusion

The **Requirement and Analysis Phase** establishes the foundation for developing the AI StudyBuddy API.

The requirement analysis identifies the main functional features such as **authentication, AI chat, note generation, quiz generation, flashcards, and progress tracking**. Non-functional requirements ensure that the API is secure, reliable, scalable, maintainable, and easy to use.

The system analysis converts these requirements into a clear technical structure involving the **frontend, REST API, AI service, authentication system, and database**.

The output of this phase can be used as the foundation for the next stages:

Requirement Analysis



System Design



Database Design



API Development



Frontend Development



Testing



Deployment